

Supercomputing for Artificial Intelligence

Foundations, Architectures, and
Scaling Deep Learning
Workloads

Supercomputing for Artificial Intelligence

Foundations, Architectures, and
Scaling Deep Learning
Workloads

Jordi Torres

WATCH THIS SPACE

BOOK SERIES - BARCELONA

2025

SUPERCOMPUTING FOR ARTIFICIAL INTELLIGENCE
Foundations, Architectures, and Scaling Deep Learning

Jordi Torres

WATCH THIS SPACE Book Series – Barcelona: Book 8

Kindle Direct Publishing

ISBN 979-831932835-9

First edition. August 2025

Cover illustration: Supercomputer Marenstrum 5

©Jordi Torres

<https://www.torres.ai>

Universitat Politècnica de Catalunya - UPC Barcelona Tech

Campus Nord, mòdul C6

Jordi Girona 1-3

08034 Barcelona

License

This book is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0). You are free to copy, distribute, and adapt portions of the work, provided that proper credit is given to the author and source, and no commercial use is made without prior authorization.

For more information on this license, please visit: <https://creativecommons.org/licenses/by-nc/4.0/>

Transparency note

AI-based tools such as Google Translate, ChatGPT, and Grammarly were used to support the writing, illustration, and language editing processes of this book. All content and figures were fully conceived, structured, and reviewed by the author.

How to cite this book

Torres, Jordi. (2025). *Supercomputing for Artificial Intelligence: Foundations, Architectures, and Scaling Deep Learning. WATCH THIS SPACE Book Series – Barcelona*. Amazon KDP. ISBN 979-831932835-9.

To my students, a constant source of inspiration.

Structure of the Book

PART I — THE INFRASTRUCTURE LAYER

1. Supercomputing Basics
2. Supercomputing Building Blocks
3. Supercomputing Software Environment and Tools

PART II — THE PARALLEL EXECUTION LAYER

4. Launching and Structuring Parallel Programs with MPI
5. GPU Programming and CUDA
6. Distributed GPU Programming

PART III — THE INTELLIGENCE ABSTRACTION LAYER

7. Neural Networks: Concepts and First Steps
8. Training Neural Networks: Basics, CNNs, and Deployment
9. Getting Started with PyTorch

PART IV — THE SCALABILITY LAYER

10. Introduction to Parallel Training of Neural Networks
11. Practical Guide to Efficient Training with PyTorch
12. Parallelizing Model Training with Distributed Data Parallel

PART V — THE LANGUAGE ABSTRACTION LAYER

13. Introduction to Large Language Models
14. End-to-End Large Language Models Workflow
15. Exploring Optimization and Scaling of LLMs
16. Looking Forward: Supercomputing and AI Futures

Table of Contents

PREFACE	19
ABOUT THIS BOOK	23
Target Audience.....	23
Approach and Philosophy	24
Structural Overview of the Book.....	25
How to Read This Book.....	28
ACKNOWLEDGMENTS	33
TECHNOLOGIES COVERED IN THIS BOOK.....	35
MAPPING TECHNOLOGIES TO BOOK CHAPTERS	37
LIST OF TASKS	39
PART I — THE INFRASTRUCTURE LAYER	43
1 SUPERCOMPUTING BASICS	47
1.1 What is Supercomputing?	47
High Performance Computing vs. Supercomputing.....	48
Parallelism: The Cornerstone of High Performance Computing	50
Understanding Parallel Computer Architectures.....	54
How Supercomputing Powers Modern Science	59
Barcelona Supercomputing Center.....	61
1.2 Performance in Supercomputing	62
Performance and Metrics	62
Standard Benchmarks in HPC.....	63
Measuring and Understanding Parallel Performance.....	68
Speedup Bounds and Scalability Models	77
1.3 Key Takeaways from Chapter 1	84
2 SUPERCOMPUTING BUILDING BLOCKS	87

2.1	System-Level Overview of a Supercomputer Architecture	87
2.2	Compute Nodes	89
	Main Components	89
	Marenostrum 5 Computer Nodes.....	93
	Accessing Marenostrum 5 Resources	99
2.3	Storage in Supercomputing	102
	Parallel File Systems in HPC	102
	Parallel File System in MareNostrum 5	105
	Working with Files on MareNostrum 5	108
2.4	Interconnection Networks in Supercomputing	111
	Bandwidth and Latency Metrics in Supercomputing	111
	Interconnection Networks in HPC Systems	112
	Interconnect Architecture in MareNostrum 5	114
2.5	Data Center Infrastructure	117
	General Concepts of HPC Data Centers.....	117
	MareNostrum 5 Data Center at BSC.....	121
2.6	Key Takeaways from Chapter 2	126
3	SUPERCOMPUTING SOFTWARE ENVIRONMENT AND TOOLS	129
3.1	Base Software Stack	129
	Foundational Software Stack	129
	Environment Modules System.....	130
	Compiling and Running C Programs.....	131
3.2	Workload Management with SLURM	137
	The Essentials of SLURM	137
	Job Directives	143
	Computing Resource Allocation.....	148
3.3	Getting Started with Docker Containers	154
	Docker basics	155
	Launching a Jupyter Notebook server inside a Docker container	160
3.4	Containerization with Singularity	162
	Containers in Supercomputing.....	162
	Building Containers	165
3.5	Key Takeaway from Chapter 3	171
PART II — THE PARALLEL EXECUTION LAYER		173
4	LAUNCHING AND STRUCTURING PARALLEL PROGRAMS WITH MPI.....	177
4.1	Foundations of Parallel Execution on Supercomputers	177
	How We Launch Parallel Jobs in This Book	177
	Why Parallel Programming Models Matter	180
4.2	Getting Started with MPI	183
	MPI Hello World.....	184
	mpirun Launcher	184

srun Launcher	186
Key MPI Concepts.....	190
4.3 Case Study: Parallelizing the Trapezoidal Rule Example	193
Serial Implementation.....	195
MPI Parallel Version.....	196
Handling I/O in MPI Programs.....	205
Barrier Synchronization.....	206
4.4 Collective Communication Primitives	207
Broadcast.....	207
Reduction Operations.....	208
Scatter and Gather.....	209
Advanced Patterns.....	210
4.5 Key Takeaways from Chapter 4	212
5 GPU PROGRAMMING AND CUDA	215
5.1 Foundations of GPU Acceleration and CUDA Programming	215
Accelerators in Supercomputing.....	215
CUDA: A Programming Platform for GPUs.....	220
Core CUDA Libraries for Accelerated Computing.....	221
5.2 Getting Started with CUDA: Threads, Kernels, and Memory	223
Hello World in CUDA.....	223
Threads Organization.....	226
Memory Management	231
Launching a CUDA Kernel.....	236
Looking Ahead: Topics Beyond the Basics	241
5.3 Case study: Matrix Multiplication in CUDA.....	243
Handling Errors	244
Managing Flattened Matrices	244
Allocating a GPU with SLURM.....	248
Timing the kernel.....	249
Performance and Scalability Analysis Using nvprof	252
5.4 Key Takeaways from Chapter 5	255
6 DISTRIBUTED GPU PROGRAMMING	257
6.1 H100: A Platform for Parallel and Distributed Computation	257
Streaming Multiprocessors (SMs).....	258
Understanding the CUDA Execution Model: Threads, Warps and Blocks.....	261
Tensor Cores: Accelerating Matrix Operations for AI	264
HBM3: Ultra-Fast On-Board Memory.....	268
6.2 High-Speed Communication for Multi-GPU Systems	269
Inter-GPU Communication Challenges in Modern AI Supercomputing	269
NVLink: Fast GPU-to-GPU Communication	270
PCI Express Gen 5: Faster CPU-GPU Communication	271
Networking for Distributed GPU Systems: RDMA and GPUDirect.....	272
Other Interconnect Technologies and Communication Protocols.....	274

6.3 Distributed Computing with CUDA-aware MPI	274
CUDA-aware MPI	275
How does CUDA-aware MPI work?	276
6.4 NCCL: Collective Communication for GPUs.....	277
Limitations of MPI for GPU-Accelerated Workloads	277
Design Principles of NCCL	278
Other Software Level Communication Libraries	279
6.5 Case Study: Distributed GPU Computing.....	280
Jacobi Algorithm.....	281
Running the Jacobi Code	284
Performance Benchmarking and Scalability Analysis	290
6.6 Key Takeaways from Chapter 6	296
 PART III — THE INTELLIGENCE ABSTRACTION LAYER	 299
7 NEURAL NETWORKS: CONCEPTS AND FIRST STEPS.....	303
7.1 The Rise of AI and the Role of Supercomputing	303
How Humans Created the First AI: The Knowledge Paradigm.....	303
How AI Began Learning from Humans: The Data Paradigm	306
When AI Learned from Experience: The Reinforcement Paradigm	308
When AI Began to Create: The Generative Paradigm	310
7.2 An Artificial Neuron	312
A Basic Deep Learning Example	312
Introduction to Basic Terminology and Notation	315
Regression Algorithms	318
A Simple Artificial Neuron	318
7.3 Neural Networks	321
Perceptron.....	322
Multilayer Perceptron.....	323
Multilayer Perceptron for Classification	326
7.4 The Softmax Activation Function.....	326
7.5 Neural Networks with TensorFlow	330
Loading Data with Keras.....	330
Input Data Preprocessing For a Neural Neuronal.....	333
Model Definition.....	335
Learning Process Configuration	338
Training the Model.....	339
Model Evaluation.....	340
Generating Predictions	342
7.6 Training Our First Neural Network.....	344
7.7 Key Takeaways from Chapter 7	347
8 TRAINING NEURAL NETWORKS: BASICS, CNNs, AND DEPLOYMENT	349
8.1 Understanding the Training Process	349

Learning Process	349
Gradient Descent	354
8.2 Parameters and Hyperparameters in Neural Networks.....	357
Model Parameterization.....	357
Hyperparameters Related to the Learning Algorithm.....	359
Activation Functions.....	362
8.3 Convolutional Neural Networks	364
Introduction to Convolutional Neural Networks	364
Building a Basic CNN in Keras.....	368
Coding Convolutional Neural Network.....	372
8.4 Training Our First Neural Network on MN5	373
8.5 Pretrained Networks and Transfer Learning	377
What is Transfer Learning?.....	377
Named Architectures and Pretrained Networks	378
A Catalog of Pretrained Models.....	382
8.6 Key Takeaways from Chapter 8	383
9 GETTING STARTED WITH PYTORCH	385
9.1 Introduction to PyTorch	386
What is PyTorch?	386
Core Components of PyTorch.....	386
Tensors in PyTorch.....	387
9.2 Neural Networks Programming in PyTorch.....	390
Importing Required Libraries	390
Loading the Dataset	391
Data Preprocessing.....	391
Defining a Neural Network Model in PyTorch	392
9.3 Configuring Neural Network Training in PyTorch.....	393
Loss Funtion	394
Optimizer	394
Backpropagation and Automatic Differentiation.....	394
9.4 Training a Neural Networks in PyTorch.....	395
Training Loop	395
Monitoring the Training Process	396
Model Evaluation in PyTorch.....	398
9.5 TensorFlow versus PyTorch.....	399
Import Required Libraries	400
Loading and Preprocessing the Data	400
Defining the Model	402
Defining the Optimizer and the Loss Function.....	402
Training the Model	402
Evaluating the Model	403
9.6 Key Takeaways from Chapter 9	405

10 INTRODUCTION TO PARALLEL TRAINING OF NEURAL NETWORKS	411
10.1 Landscape of Parallel Deep Learning Frameworks.....	412
The Challenge of Communication Across GPUs.....	412
High-Level Frameworks for Distributed Deep Learning.....	413
10.2 Comparing CPU and GPU Performance: A Practical Training Case	416
Training Data: CIFAR10	416
Model architectures: ResNet	418
Baseline Code: Sequential Training on CPU and GPU	420
10.3 Accelerate Training with Parallelism in TensorFlow	423
Types of Parallelism.....	423
Parallel Training with TensorFlow.....	426
Exploring the Parallelization of the Training Step with MirroredStrategy.....	432
10.4 Impact of Model Size on Parallel Training	441
Case Study: ResNet152	441
Parallelization of the ResNet152V2 Neural Network.....	441
Comparing ResNet50 vs ResNet152V2	445
Applying Amdahl's and Gustafson's Laws.....	450
10.5 Key Takeaways from Chapter 10	453
11 PRACTICAL GUIDE TO EFFICIENT TRAINING WITH PYTORCH	455
11.1 Introduction.....	455
11.2 Illustrative Case Study	456
Custom Datasets	456
Models: ViT and Custom	458
Code Walkthrough: The train.py Script.....	460
11.3 Setting up for Experiment Execution	464
Python code outputs.....	465
SLURM Job Submission Scripts	466
11.4 Tuning the Maximum Batch Size Hyperparameter	470
Motivation: Why Does Batch Size Matter?	470
Methodology: Empirical Search	471
Experimental Results	471
11.5 Efficient Data Handling: Preventing DataLoader Bottleneck	473
Observing the Bottleneck with a Lightweight Model	474
Increasing num_workers to Unlock Performance	476
Returning to ViT: When DataLoader Parallelism Is Hidden	479
11.6 Mixed Precision and Tensor Cores.....	481
Understanding Floating-Point Formats	481
Mixed Precision Training with AMP	482
Experimental Results	483
11.7 Torch Compile	484
11.8 Enhancing Throughput with Minimal Code Changes.....	485

11.9 Key Takeaways from Chapter 11	487
12 PARALLELIZING MODEL TRAINING WITH DISTRIBUTED DATA PARALLEL	489
12.1 How Computation is Distributed.....	490
Understanding the Full Stack of Distributed Training	490
What is torchrun?	491
What is Distributed Data Parallel?	493
12.2 Dataset Sharding and Preprocessing in DDP.....	497
12.3 Setting Up for DDP Experiment Execution.....	498
Code Walkthrough: Key Differences in train_ddp.py	499
Slurm Job Submission Scripts	502
12.4 Throughput and scalability measurements	505
Results with micro-224 Dataset.....	505
A Brief Analytical Reflection.....	508
Results with the Tiny Dataset	510
Critical Thinking for an HPC practitioner	512
12.5 Key Takeaways from Chapter 12	517
 PART V — THE LANGUAGE ABSTRACTION LAYER	 519
13 INTRODUCTION TO LARGE LANGUAGE MODELS.....	523
13.1 The Evolution of LLMs and the Central Role of Supercomputing	523
A New Era of Artificial Intelligence Powered by Large Language Models	523
Large-Scale Pretraining: Building the Foundation with Data and Compute	524
Post-Training and Alignment: Teaching Models to Follow Human Instructions	525
Advanced Inference: Real-Time Deployment at Global Scale.....	526
Autonomous Agents: The Emerging Frontier of LLM Applications	528
Supercomputing as the Silent Engine Behind LLM Progress.....	529
13.2 The Anatomy of Large Language Models	530
Transformers: The Foundation of Modern LLMs.....	530
Transformer Variants: Encoder, Decoder, and Encoder–Decoder.....	533
What Is a Language Model?	535
Transfer Learning: The Two-Stage Training Paradigm	536
The Attention Mechanism: How Transformers Understand Context	538
13.3 Hugging Face Framework.....	542
Why Hugging Face Matters for LLMs.....	543
Core Components of the Hugging Face Ecosystem.....	545
Hugging Face Trainer API: Simplifying Distributed Training.....	549
13.4 Key Takeaways from Chapter 13	555
14 END-TO-END LARGE LANGUAGE MODELS WORKFLOW	557
14.1 Understanding the LLM Workflow.....	557
14.2 Authentication and Downloading Models with Hugging Face.....	559
12.2.1 Authenticating in Hugging Face	559

Downloading from Hugging Face	562
14.3 LLM "Hello World" in Google Colab	563
Inference with a Pretrained Model	564
Fine-tuning a model With a Toy Dataset	565
14.4 Preparing Models and Datasets for Supercomputing	568
Download and Transfer of the Model	568
Download and Transfer of the Dataset	569
14.5 LLM "Hello World" on Marenstrum 5	570
Python Script for Inference and Fine-Tuning	571
SLURM Script and Explanation	573
Remarks on Simplified Inference	575
14.6 Key Takeaways from Chapter 14	576
15 EXPLORING OPTIMIZATION AND SCALING OF LLMs	577
15.1 Case study and code	578
LLaMA 3.2 with 1B parameters	578
Synthetic Data	578
Baseline Training Script	578
Performance Metrics: Measuring Training Efficiency	580
SLURM Job Submission Script	581
Running the Baseline Experiment	581
15.2 Efficient Training on a single GPU	582
Batch Size Scaling	582
Enhancing Training Efficiency with Mixed Precision	583
Fine-Tuning Model Precision	584
Enhancing Attention Mechanisms	585
Accelerating Training with Triton and Liger Kernels	586
15.3 Scaling Up: Distributed Training Across Multiple GPUs	588
Experimental Setup	589
Scaling Results	590
15.4 Cost-Efficient Optimization: Performance per Euro	591
15.5 Key Takeaways from Chapter 15	594
16 LOOKING FORWARD: SUPERCOMPUTING AND AI FUTURES	597
16.1 The Eternal Triad: Data, Algorithms, and Compute	598
16.2 Megascale Hardware and the Data Center Era	599
16.3 Energy Becomes the New Bottleneck	600
16.4 Quantum Supercomputing: Hype or Hope?	601
16.5 Beyond Big Data: Toward the Era of Experience	602
16.6 From Robots to Embodied Data	603
16.7 Key Takeaways from Chapter 16	604

17 APPENDICES.....	609
17.1 Essential Linux Commands	609
17.2 SSH Public Key Authentication.....	611
17.3 C Language Basics.....	612
17.4 Taking Timing	616
17.5 Python and Its Libraries.....	620
17.6 Jupyter Notebook Basics	640
17.7 GLOSSARY	645
A	645
B	645
C	646
D	647
E	648
F	648
G	649
H	649
I	650
J	650
K	650
L	651
M	651
N	652
O	653
P	653
Q	654
R	654
S	654
T	655
U	656
V	656
Z	656

Preface

Like most things that truly matter in life, this book has been a long journey in the making. Let me tell you how it came to be.

Parallelism

From my earliest days as a student, I was fascinated by parallelism — the beating heart of any supercomputer — a concept I fell in love with for both its elegance and its potential. I suppose the context helped: I studied at the Facultat d'Informàtica de Barcelona (FIB), where I had the chance to learn from professors affiliated with the Computer Architecture Department at Universitat Politècnica de Catalunya (UPC Barcelona Tech). They not only taught us technical content, but also conveyed something deeper — how important it was that we, their students, could enjoy those early years of academic freedom, hard-won by previous generations.

I knew exactly who I wanted to work with. As a student, I worked hard to join that department, and eventually the opportunity came: I began collaborating with the research team led by Professor Jesús Labarta and soon after started my PhD under the supervision of Professor Eduard Ayguadé. At the time, we were all working under the umbrella of the CEPBA (European Center for Parallelism of Barcelona), which would later become the foundation of what is now the Barcelona Supercomputing Center (BSC). The CEPBA was led by Professor Mateo Valero — a name surely familiar to anyone reading this book. He was the driving force behind this passionate and visionary department in parallelism, and his support and friendship from those early days have had a profound and lasting impact on my career.

Machine Learning

After many years of work focused on the foundations of supercomputing, in the early 2000s I started to explore how supercomputing could contribute to machine learning. I was encouraged in this by my long-time colleague and good friend Professor Ricard Gavalda, a recognized expert in the field. This led, in 2006, to exciting collaborations with brilliant PhD students like

Nicolas Poggi and Josep Lluís Berral. Together, we explored the intersection of machine learning and supercomputing — a topic that would shape the next decades of my career.

Since then, I've continued studying and deepening my understanding of machine learning through personal effort and continuous learning. One of the true privileges of academic life is that studying — learning — is part of the job. I've taken specialized courses, read foundational textbooks carefully, and followed the evolution of the field through key scientific papers.

Deep Learning

Allow me to pause and share a few moments that truly thrilled me — milestones that made me feel the pulse of the emerging deep learning revolution and what lies beneath today's AI systems.

In 2014, thanks to the encouragement of my colleague Jordi Nin, I was first exposed to deep learning just as it was beginning to gain traction. I was deeply impressed to see how it was transforming entire research areas — all thanks to the power of GPUs, which we in the supercomputing world had long used for numerical simulations.

In 2015, Dr. Joan Capdevila, then a PhD student, suggested me to join him at the UPM Machine Learning and Advanced Statistics Summer School. There, I took a course on neural networks and deep learning that left a strong mark on me. I met exceptional researchers like Professors Concha Bielza and Pedro Larrañaga, and for the first time, I saw how my research focus could shift.

One year later, on the advice of Dr. Oriol Vinyals, I attended the Machine Learning Summer School — that year held in Cádiz. It was an internationally renowned event bringing together some of the world's leading experts in machine learning. For me, it was also the first time I encountered reinforcement learning — a key piece in today's AI models. My mind was buzzing. Despite being one of the older attendees —surrounded by younger researchers— I felt grateful and inspired. I knew I had to make a leap and fully commit to this new direction in my work.

Artificial Intelligence

In 2016, I began co-supervising doctoral theses at the intersection of AI and supercomputing with the visionary Dr. Xavier Giró-i-Nieto. Working alongside PhD students like Miriam Bellver, Víctor Campos, and Amanda Duarte gave me fresh insight into modern AI and, without a doubt, helped shift the BSC's research direction in this field.

This stage also allowed me to actively participate in top-tier AI conferences — quite a contrast from the supercomputing conferences I had been attending for decades. I had the chance to connect with the deep learning community, and in some cases, to co-author papers with brilliant researchers: Ferran Marquès, Cristian Canton-Ferrer, Amaia Salvador, Kevin McGuinness, Noel E. O'Connor, Kevis-Kokitsi Maninis, Jordi Pont-Tuset, Luc Van Gool, Shih-Fu Chang, Santiago Pascual, Eva Mohedano, Shruti Palaskar, Lucas Ventura, Deepti Ghadiyaram, Florian Metze, Alexander Trotter, Caiming Xiong, and Richard Socher. I consider myself very fortunate to have learned from all of them, and I am truly grateful for their generosity and collaboration.

This Book

Thanks to this long path, I was already deeply immersed in the deep learning field when large language models (LLMs) began to take off. And perhaps for that reason, coming from a supercomputing background, it felt natural to recognize the real driving force behind this new AI revolution: supercomputing itself.

This book was born out of a specific need: to provide a modern and rigorous theoretical resource for my courses at the Universitat Politècnica de Catalunya (UPC), tailored to the current challenges of supercomputing for AI. For years, I searched for a textbook that bridged supercomputing and deep learning — something pedagogical, cohesive, and practical — but never found one.

So I decided to write it.

I've been lucky to teach with MareNostrum supercomputers family right next door — quite literally, as it's located on the UPC campus. Teaching parallel programming, distributed training, or performance optimization in

such an environment is a privilege.

Over the years, I've been writing and refining teaching material for various courses — which have gradually converged toward what we now call HPC for AI. When the FIB invited me to launch a course with that name, it felt like the right time to gather, organize, and formalize everything into this book. I've finally completed it in the 2025–2026 academic year, and now you have it in your hands.

This book is not just a tool for my courses. It is an invitation to understand why, now more than ever, artificial intelligence is —at its core— a supercomputing problem.

About This Book

Target Audience

As mentioned in the epilogue, this book emerged from a specific academic need: to provide my courses at the Universitat Politècnica de Catalunya (UPC) with a modern and rigorous resource tailored to the evolving challenges of supercomputing for artificial intelligence.

In my own teaching practice, selected chapters or adapted excerpts from this book are shared as official course material with students at the UPC. This ensures that the core content reaches its primary audience—those for whom it was initially developed—as part of their academic training in AI and supercomputing.

Although it was originally conceived as supporting material for students enrolled in AI-focused supercomputing courses, the content has been carefully structured to serve a broader audience. Its modular design, hands-on examples, and real-world case studies make it accessible and relevant to:

- Master’s and advanced undergraduate students seeking practical knowledge of HPC applied to AI;
- Researchers and academics interested in the computational foundations of large-scale AI training and inference;
- Engineers and data scientists optimizing deep learning workloads on distributed infrastructures;
- HPC practitioners and system architects aiming to understand how AI workloads interact with modern hardware and software stacks;
- Instructors and educators searching for up-to-date teaching material or inspiration in the fields of HPC and AI systems.

To support this diversity of profiles, the book adopts a progressive and modular approach, allowing readers to engage with the material at different entry points depending on their background and objectives. Educators, in particular, will find a rich set of reproducible experiments, diagrams, visual

explanations, and practical exercises that can be directly incorporated into coursework or adapted to various instructional contexts. The chapter structure and terminology mapping have been carefully curated to facilitate course planning and topic sequencing.

This is not intended to be a comprehensive or encyclopedic treatise—such an endeavor would be unrealistic given the pace at which the field evolves. Instead, the book focuses on a well-curated selection of core concepts and tools, directly inspired by classroom teaching and complemented by selected topics that offer both technical depth and narrative coherence.

The multifaceted approach—bridging theory and practice, and spanning multiple levels of the AI supercomputing stack—is intended to make this book a valuable companion not only for students, but also for professionals, educators, and innovators navigating the rapidly evolving landscape of large-scale AI computing.

Approach and Philosophy

Beyond its technical scope, this book reflects a commitment to teaching through doing, a principle that has guided the design of every chapter. Each concept is reinforced with hands-on tasks that can be executed on real supercomputing systems or cloud-based environments like Google Colab. This deliberate integration of theory and practice aims to help readers not only understand the “what” and “why,” but also confidently navigate the “how” of deploying AI workloads in distributed, high performance environments.

Reproducibility and clarity were core priorities during the writing process. All examples have been tested and aligned with real software environments, such as MareNostrum 5 at the Barcelona Supercomputing Center. This ensures that students and practitioners alike can replicate the experiments and build upon them using state-of-the-art tools like SLURM, PyTorch, Hugging Face, or CUDA. These reproducible building blocks empower readers to adapt and scale solutions to their specific contexts.

The book also reflects the author’s dual experience as a researcher and educator. With more than tree decades in both domains, the goal was not only to transfer knowledge, but to bridge the gap between academic learning

and technological application. Readers will find frequent annotations, side notes, and discussions that make explicit the rationale behind architectural decisions or implementation choices—encouraging critical thinking and independent exploration.

Structural Overview of the Book

This book spans a wide range of topics—hardware, system software, parallel programming, deep learning, distributed training, and modern AI models. While this broad scope reflects the reality of AI supercomputing workflows, it also poses a challenge for learners: how can we make sense of so many technologies without getting lost in the details?

The content of this book is structured following a hierarchical abstraction model—a layered stack. Each layer isolates a specific level of the supercomputing ecosystem, allowing students to build their understanding progressively without being overwhelmed by the intricacies of lower or higher levels prematurely. This structure is intended to address the inherent complexity of the field in a pedagogical and accessible way, focusing on what is essential for mastering the connection between supercomputing and artificial intelligence today.

We will start (in Part I) with a classically styled presentation of traditional supercomputing topics — the kind that has formed the backbone of supercomputing courses for many years: their purpose, architecture, performance metrics, software systems, etc. Then (in Part II), we will describe how to execute parallel programs with traditional parallel programming models on a supercomputer.

From that point onward (in Parts III, IV, and V), we shift toward the current frontiers of the field. These sections explore how supercomputing is now used to address emerging demands posed by artificial intelligence. To illustrate this, we present concrete, real-world examples executed on MareNostrum 5, the flagship supercomputer of the Barcelona Supercomputing Center (BSC). These hands-on cases are accompanied by the necessary theoretical context, discussing both the nature of each problem and how it is solved on a modern supercomputing platform.

This progressive approach ensures that each chapter builds upon the

foundations laid by the previous ones, enabling a gradual and coherent development of knowledge. In this way, students can navigate the multifaceted landscape of supercomputing for AI with clarity and purpose.

The Infrastructure Layer

This bottom level encompasses the fundamental knowledge of computer hardware and data center infrastructure. It also includes the system software and low-level environment that underpin a supercomputer, such as the operating system, compilers, or drivers. Additionally, this layer covers the job scheduler SLURM responsible for orchestrating the allocation of computing resources across the cluster. Optionally, a containerization subsystem may be integrated at this level to encapsulate applications and their dependencies, ensuring consistent and portable runtime environments across different supercomputing platforms.

The Parallel Execution Layer

In this part, we shift from the foundational infrastructure discussed in Part I to the practical orchestration of parallel programs on a supercomputer. This is a crucial intermediate step between understanding the system and running large-scale AI workloads, and it forms the operational core of any well-rounded course in high performance computing.

We begin by introducing `srn`, the central tool used to launch parallel jobs in SLURM-managed environments. This launch mechanism is the gateway to scalable execution and is used consistently throughout the book.

We show traditional parallel programming models, such as MPI and CUDA, to accelerate computation by exploiting parallelism—the key ingredient that truly empowers supercomputing.

This layer provides the student with a solid understanding of how to launch parallel programs on a supercomputer—laying the groundwork for the deep learning workflows covered in the following parts of the book.

The Intelligence Abstraction Layer

In Part III, we introduce the foundational concepts of deep learning along with the software frameworks that have become standard tools for developing modern AI applications. These frameworks, such as TensorFlow and

PyTorch, provide a high-level abstraction that simplifies the design, training, and deployment of neural networks—removing the need to manually manage the low-level parallel code we encountered in the previous part on parallel execution.

Through practical examples based on TensorFlow we illustrate how to implement and train AI models. This part serves as an entry point to deep learning theory for students who may be approaching these concepts for the first time.

The Scalability Layer

The Part IV of this book is devoted to the study of efficient scaling strategies necessary for training large-scale neural networks across multiple nodes of a supercomputing infrastructure. The content is structured around practical case studies developed using TensorFlow and PyTorch framework. Through these exercises, students will explore techniques for parallelizing training across multiple GPUs and will become familiar with launcher tools designed to orchestrate and manage distributed training workloads effectively.

The Language Abstraction Layer

The highest level of the abstraction hierarchy presented in Part V of this book is dedicated to the study and practical use of Large Language Models (LLMs), which currently constitute one of the most influential developments in artificial intelligence. This part introduces the essential theoretical concepts of LLMs, ensuring that students without prior background in the field can fully engage with the material. At this level, interaction with AI models is centered around standardized frameworks and pre-trained solutions, and the complexities of the underlying hardware, distributed architectures, and training algorithms are almost completely abstracted away. The Hugging Face Transformers library exemplifies this paradigm by offering standardized model architectures, access to large pre-trained models, and simple APIs for fine-tuning and deployment.

Together, these five parts form a coherent journey through the supercomputing landscape, equipping students with both theoretical knowledge and practical skills needed to drive today's AI advances. Figure 2 summarizes this layered architecture.

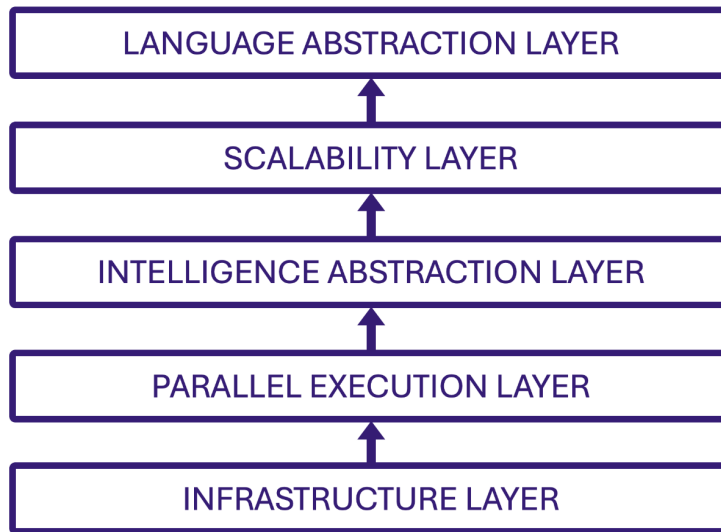


Figure 2 – Hierarchical structure of the book, illustrating the five abstraction layers that guide the progressive learning path from supercomputing fundamentals to Large Language Models.

How to Read This Book

Can I Skip the Tasks?

Absolutely. This book has been written to support multiple styles of reading and learning. You are welcome to read it as a theoretical and conceptual guide without running any code or completing the practical tasks. The chapters are designed to convey the key ideas clearly, even if you are not actively working on a supercomputer or executing scripts.

In the courses I teach using this book as a reference, each subject will include clear guidance on which tasks to complete and how to submit responses, as part of the course-specific instructions.

Typographical Conventions Used in This Book

Throughout this book, we use a set of typographical conventions to improve readability and clarity. These are summarized below for your reference:

- Text in `Courier` is used to indicate variable names in code, file names, URLs, and similar elements.
- Text in *italics* is used to highlight important concepts within the book's content, often when they are introduced for the first time.
- Code blocks are displayed using a monospaced font on a gray background, as shown below:

```
#include <stdio.h>
int main(){
    printf("Hello world!\n");
}
```

- Highlighted lines within code blocks—those that are referenced in the main text—appear in bold monospaced font on the same gray background:

```
#include <stdio.h>
int main(){
    printf("Hello world!\n");
}
```

- Command-line commands are shown in a monospaced font on a gray background, prefixed with a \$ symbol to indicate they are entered in a terminal session:

```
$ module load intel
$ icx hello.c -o hello
```

- Standard output is shown in `Courier New` font:

```
$ ./hello
Hello world!
```

Where to Get the Code and Errata

All source code, job scripts, and configuration files referenced in the book are available in the public GitHub repository:

<https://github.com/jorditorresBCN/HPC4AIbook>

This repository also provides information on how to access updates and report errata.

While every effort has been made to ensure the accuracy and clarity of the content, a significant part of this book—especially the sustained effort over the past year, culminating in an intense final sprint—was completed in personal time, outside my usual academic and research responsibilities at the UPC and the BSC. During this period, I remained fully committed to my other responsibilities, which naturally limited the time available for writing. Even so, the book was shaped with the best intentions to support students and professionals in the field. Inevitably, there may be errors or oversights, for which I sincerely apologize. I appreciate readers’ understanding, and I see this as a living book: I will continue updating the GitHub repository with any corrections or suggestions, and integrate improvements into future editions.

For any questions, suggestions, or to report errors directly, you may also contact the author via email at: HPC4AIbook@gmail.com

Disclaimer and Author’s Note

This book is intended for educational and research purposes only. While it includes practical examples and code, the author assumes no responsibility for any misuse, misinterpretation, or consequences arising from the application of the material in real-world systems or production environments. Use the content at your own risk and always validate your implementations.

Throughout the book, readers may notice that certain transversal topics appear across multiple chapters. This is intentional. Over the years, I have shared selected chapters of this book as teaching materials for different university courses, depending on each course’s specific focus. While the book as a whole covers a broad range of themes, I found it more coherent—and more useful for the reader—to consolidate everything into a single volume, rather than fragment the content across several separate textbooks. Of course, each course targets different profiles: some students come from an

HPC background, others from AI or cloud computing. The result is a book that reflects the diversity of those contexts while maintaining a unified pedagogical vision.

Additionally, the content of this book builds upon materials I have developed over many years for various courses and training programs. As a result, readers may notice some stylistic differences between chapters. I apologize for any inconsistencies or rough edges. With over 650 pages, perfection is difficult to achieve—but the intent is always to provide useful, practical guidance.

I will continue refining and updating the book with each academic year. Your feedback is not only welcome—it is essential.

Thank you!

Acknowledgments

Many people have helped me—some more, some less—over the years with the content of the notes and previous books that made this one possible. To all of them, thank you:

Ferran Agulló, Pablo Arancibia, Oriol Aranda, Gonzalo Artiach, Roser Bellido, Enric Aromí, Eduard Ayguadé, Guifré Ballester, Javier Bartolomé, Yolanda Becerra, Jeroni Boixareu, Míriam Bellver, Miguel Bernabeu, Josep Lluís Berral, Víctor Campos, Ramon Canal, Daniel Cano, Joan Capdevila, Manuel Carbonell, David Carrera, Oliver Chan, Miquel Escobar Castells, Mauro Cavaller, Hugo Centeno, François Chollet, Juan Luís Domínguez, Amanda Duarte, Mauricio Echevarría, Rosa Maria Esqué, Agustín Fernández, Javier Ferrando, Joan Gallet, Raul Garcia Fuentes, David Garcia, Raul Garcia, Bernat García, Pol Garcia Recasens, Ricard Gavaldà, Sergi Girona, Xavier Giró-i-Nieto, Alberto Gutiérrez, Andrés Gómez, Antonio Guardia, Jordi Guitart, Laura Juan, Álvaro Jover Álvarez, Ferran Julià, Jiri Kraus, Jesús Labarta, Isabel Larraburu, Josep M. Martorell, Xavier Marturià, Màrius Mollà, Jordi Morera, Jordi Nadal, Nacho Fernández, Nacho Navarro, Jon Navarro, Jordi Nin, Ramon Nou, Oriol Núñez, Nico Poggi, Alberto Pou, Oriol Pujol, Félix Ramos, Oscar Romero, Sergi Sales, Xisco Sastre, Fernando García Sedano, Antoni-Joan Solergibert, Alejandro Fernández Suárez, Laia Tarrés, Joan Teruel, Bernat Torres, Júlia Torres, Jordi Torres i Mas, Rubèn Tous, Carlos Tripiana, Mateo Valero, David Vicente, Alba Vila, Oriol Vinyals, Katy Wallace, Maurici Yagües.

I also want to extend my thanks—and my apologies—to all those I've surely forgotten to mention.

Technologies Covered in This Book

Development Tools:

Jupyter, Colab

LLM Models:

LlaMA 3.2, opt-1.3b

LLM Libraries:

Hugging Face (Trainer, Transformers, Datasets, Tokenizers)

AI Frameworks:

PyTorch, TensorFlow

Model Execution Optimizations

Flash Attention, Liger kernels, Mixed Precision, Model Precision

Distributed Runtimes & Libraries:

DDP, accelerate

Parallel Launchers:

torchrun, srun, mpirun

Communication Middleware:

MPI, NCCL, GPUDirect

Programming Languages & Compilers:

Python, C/C++, CUDA, gcc, icx

OS & Resource Managers:

Linux, SLURM, Singularity, Dockers

Hardware:

CPUs, GPUs (H100), Interconnect (NVLink, Infiniband)

Mapping Technologies to Book Chapters

Category	Technology	Chapters
Development Tools	Jupyter	Cap. 3,7, Appendices
Development Tools	Colab	Cap. 7, 14, Appendices
LLM Models	LLaMA 3.2	Cap. 14
LLM Models	opt-1.3b	Cap. 13
LLM Libraries	Hugging Face	Cap. 14
LLM Libraries	Trainer	Cap. 14
LLM Libraries	Transformers	Cap. 14
LLM Libraries	Datasets	Cap. 14
LLM Libraries	Tokenizers	Cap. 14
AI Frameworks	PyTorch	Cap. 9, 11, 12
AI Frameworks	TensorFlow	Cap. 7, 8, 10
Model Execution Optimizations	Flash Attention	Cap. 15
Model Execution Optimizations	Liger kernels	Cap. 15
Model Execution Optimizations	Mixed Precision	Cap. 11, 15
Model Execution Optimizations	Model Precision	Cap. 15
Distributed Runtimes & Libraries	DDP	Cap. 12, 15
Distributed Runtimes & Libraries	accelerate	Cap. 13
Parallel Launchers	torchrun	Cap. 12, 15
Parallel Launchers	srun	Cap. 4, 5, 11, 12, 13

Parallel Launchers	mpirun	Cap. 4 , 6
Communication Middleware	MPI	Cap. 4, 6
Communication Middleware	NCCL	Cap. 6, 12
Communication Middleware	GPUDirect	Cap. 6
Programming Languages & Compilers	Python	Appendices
Programming Languages & Compilers	C/C++	Appendices
Programming Languages & Compilers	CUDA	Cap. 5
Programming Languages & Compilers	gcc	Cap. 3
Programming Languages & Compilers	icx	Cap. 3
OS & Resource Managers	Linux	Appendices
OS & Resource Managers	SLURM	Cap. 3
OS & Resource Managers	Singularity	Cap. 3
OS & Resource Managers	Dockers	Cap. 3
Hardware	CPUs	Cap. 2
Hardware	GPUs (H100)	Cap. 2, 6
Hardware	Interconnect (NVLink)	Cap. 2,6
Hardware	Interconnect (Infiniband)	Cap. 2

List of Tasks

TASK 1.1 – PLAYING WITH SPEEDUP AND EFFICIENCY	73
TASK 1.2 – UNDERSTANDING STRONG AND WEAK SCALING	75
TASK 1.3 – APPLYING AMDAHL’S LAW	80
TASK 1.4 – GUSTAFSON-BARSIS’S LAW AND SCALING INTUITION	83
TASK 2.1 – LOG INTO MARENOSTRUM 5.....	101
TASK 2.2 – CHANGE YOUR PASSWORD	101
TASK 2.3 – (OPTIONAL) ENABLE PASSWORDLESS SSH AUTHENTICATION	101
TASK 2.4 – TRANSFER FILES USING SCP	110
TASK 2.5 – (OPTIONAL) MOUNT THE MN5 FILESYSTEM ON YOUR LAPTOP	110
TASK 3.1 – COMPARE ICX AND GCC COMPILER OPTIMIZATIONS	137
TASK 3.2 – REFLECTING ON SLURM JOB PRIORITIZATION	143
TASK 3.3 – SUBMIT YOUR FIRST SLURM JOB.....	146
TASK 3.4 – INSTALL DOCKER IN YOUR PLATFORM.....	157
TASK 3.5 – DOWNLOAD DOCKER IMAGE	158
TASK 3.6 – RUN DOCKER IMAGE.....	159
TASK 3.7 – STOP A DOCKER CONTAINER.....	159
TASK 3.8 – RUN DOCKER WITH PORT MAPPING.....	161
TASK 3.9 – START THE JUPYTER NOTEBOOK SERVER	161
TASK 3.10 – CREATE AND RUN A TEST NOTEBOOK	162
TASK 4.1 – COMPILE AND RUN YOUR FIRST MPI PROGRAM	187
TASK 4.2 – OBSERVE NODE DISTRIBUTION USING HOSTNAMES	189
TASK 4.3 – POINT-TO-POINT COMMUNICATION	193
TASK 4.4 – WRITE AND RUN THE SEQUENTIAL PROGRAM THAT ESTIMATE π	196
TASK 4.5 – WRITE AND RUN THE PARALLEL MPI CODE TO ESTIMATE THE VALUE OF π	199
TASK 4.6 – ANALYSIS USING GUSTAFSON’S LAW TO ESTIMATE THE VALUE OF π	205
TASK 4.7 – EXPERIMENTING WITH SCATTER AND GATHER	210
TASK 5.1 – YOUR FIRST HELLO WORLD IN CUDA.....	226
TASK 5.2 – DIMENSIONALITY OF A THREAD BLOCK AND GRID.....	231
TASK 5.3 – INVESTIGATING PARALLEL EXECUTION WITH MULTIPLE THREADS	235
TASK 5.4 – ELEMENT-WISE VECTOR ADDITION USING CUDA	241
TASK 5.5 – PARALLEL MATRIX MULTIPLICATION WITH CUDA.....	247
TASK 5.6 – RUNNING CUDA JOBS WITH SLURM	249
TASK 5.7 – PROFILING MATRIX MULTIPLICATION ON THE GPU.....	252
TASK 5.8 – COMPUTE-BOUND VS MEMORY-BOUND	254

TASK 6.1 – REFLECTING ON CUDA'S EXECUTION MODEL	264
TASK 6.2 – PRECISION TRADE-OFFS: TRUE OR FALSE?	267
TASK 6.3 – SUBMIT AND VALIDATE THE FIRST PERFORMANCE RUN	286
TASK 6.4 – UNDERSTAND THE METRICS COLLECTION	287
TASK 6.5 – EXPLORE THE EFFECT OF OPTIMIZED COMPILATION FLAGS	288
TASK 6.6 – EVALUATE THE IMPACT OF THE CUB LIBRARY	290
TASK 6.7 – BENCHMARKING THE IMPACT OF GPU COUNT AND PROBLEM SIZE	295
TASK 7.1 – SET UP YOUR GOOGLE COLAB ENVIRONMENT	345
TASK 7.2 – EXECUTE THE PROVIDED NOTEBOOK STEP-BY-STEP	346
TASK 7.3 – IMPROVE THE ACCURACY OF YOUR MODEL	347
TASK 8.1 — IMPROVING A BASIC CNN MODEL	373
TASK 8.2 — EXPORTING THE PYTHON SCRIPT	374
TASK 8.3 — RUNNING YOUR FIRST NATURAL NETWORK ON A LOGIN NODE	375
TASK 8.4 — SUBMITTING YOUR FIRST GPU DL TRAINING WITH SLURM	377
TASK 9.1 – COMPARATIVE IMPLEMENTATION IN PYTORCH AND TENSORFLOW	405
TASK 10.1 – TASK SETUP AND FILE STRUCTURE	419
TASK 10.2 – CODE REVIEW AND UNDERSTANDING	421
TASK 10.3 – TRAINING ON CPU (BASELINE PERFORMANCE)	422
TASK 10.4 – GPU EXECUTION TIME AND CPU VS GPU COMPARISON	422
TASK 10.6 – ANALYZE THE IMPACT OF GPU PARALLELISM	440
TASK 10.7 – PARALLELIZATION OF RESNET152	445
TASK 10.8 – COMPARING PARALLEL TRAINING PERFORMANCE ACROSS MODEL SIZES	448
TASK 10.9 – RESNET101V2 SCALABILITY ANALYSIS	449
TASK 10.10 – INTERPRETING RESULTS WITH AMDAHL'S AND GUSTAFSON'S LAWS	452
TASK 10.11 – APPLYING AMDAHL'S AND GUSTAFSON'S LAWS TO RESNET101V2	453
TASK 11.1 – FIND THE MAXIMUM VIABLE BATCH SIZE	473
TASK 11.2 – INVESTIGATING THE DATALOADER BOTTLENECK WITH A LIGHTWEIGHT MODEL	476
TASK 11.3 – OPTIMIZING DATALOADERS WITH APPROPRIATE NUM_WORKERS	478
TASK 11.4 – CONFIRMING DATALOADER EFFICIENCY WITH ViT	480
TASK 11.5 – ENABLE MIXED PRECISION WITH ViT + MICRO-224	483
TASK 11.6 – REPRODUCING THE EFFECT OF TORCH.COMPILE()	485
TASK 11.7 – REPORT YOUR CONCLUSIONS	485
TASK 11.6 – MINOR CODE TWEAKS FOR A FINAL THROUGHPUT BOOST	486
TASK 12.1 – REPRODUCING DISTRIBUTED TRAINING RESULTS ON MN5	509
TASK 12.2 – ANALYZE AND COMPARE SCALING EFFICIENCY	516
TASK 12.3 – INVESTIGATE DIMINISHING RETURNS IN TRAINING TIME	517
TASK 12.4 – FIND THE SWEET SPOT FOR YOUR USE CASE	517

TASK 14.1 – OBTAIN YOUR HUGGING FACE ACCESS TOKEN	562
TASK 14.2 – DOWNLOAD AND RUN THE HELLO WORLD IN GOOGLE COLAB.....	567
TASK 14.3 – DOWNLOAD THE MODEL AND DATASET LOCALLY USING HUGGINGFACE-CLI	570
TASK 14.4 – TRANSFER THE MODEL AND DATASET TO MARENOSTRUM 5.....	570
TASK 14.5 – RUN THE INFERENCE AND FINE-TUNING SCRIPT ON MN5.....	574
TASK 14.6 – COMPARE EXECUTION IN COLAB VS MN5.....	574
TASK 15.1 – BASELINE EXPERIMENT USING FACEBOOK/OPT-1.3B	582
TASK 15.2 – FINDING THE OUT-OF-MEMORY LIMIT	583
TASK 15.3 – MIXED PRECISION TRAINING.....	584
TASK 15.4 – MODEL PRECISION	585
TASK 15.5 – INCREASING BATCH SIZE WITH MODEL PRECISION.....	585
TASK 15.6 – ENABLING FLASH ATTENTION	586
TASK 15.7 – INCREASING BATCH SIZE WITH FLASH ATTENTION	586
TASK 15.8 – USING THE LIGER KERNEL.....	587
TASK 15.9 – AUGMENTING BATCH SIZE DUE TO LIGER KERNELS.....	588
TASK 15.10 – SCALING ON MULTIPLE GPUS	591
TASK 15.11 – FINAL REFLECTIONS.....	593
TASK 16.1 — MAPPING THE PENDULUM SHIFTS OF THE TRIAD	598
TASK 16.2 — EXPLORING THE NEW GIANTS OF COMPUTE	599
TASK 16.3 — POWERING THE FUTURE OF AI.....	600
TASK 16.4 — CHARTING THE ROLE OF QUANTUM IN FUTURE SUPERCOMPUTING	601
TASK 16.5 — FROM DATA FARMS TO AI PRESERVES	602
TASK 16.6 — EMBODIMENT AS A SOURCE OF LEARNING	603